

Capítulo 1: Entradas y Salidas

Objetivo

Comprender el funcionamiento de los registros asociados a las entradas y salidas del microcontrolador CH32V003, así como también conocer y usar el IDE MounRiver Studio 2 de la empresa WCH.

Desarrollo Teórico

1. MounRiver Studio 2

MounRiver Studio 2 es un entorno de desarrollo que proporciona las herramientas necesarias para programar algunas familias de microcontroladores WCH, el IDE permite usar el lenguaje de programación C/C++ y también ASM (ensamblador) en dado caso que se requiera una mayor velocidad en la manipulación de los datos, en su última actualización se implementó el uso de arquitecturas ARM por lo cual se podrá trabajar con arquitectura RISC-V y ARM.

1.1 Creación de proyecto

Para la creación del primer proyecto es necesario en primera instancia tener instalado [MounRiver Studio 2](#), posteriormente deberá seguir los siguientes pasos:

1. En la ventana principal deberá dar clic izquierdo en File (Recuadro Rojo), posteriormente en New y MounRiver Project (Recuadros azules) como se muestra en la imagen 1, cabe mencionar que hay una opción directa en la ventana principal de MounRiver Studio con el nombre de Create MounRiver Project.

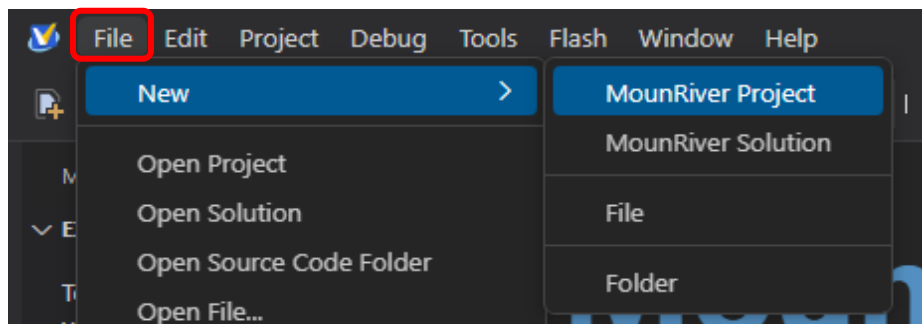


Imagen 1. Pestaña para crear proyecto.

2. Al realizar el paso anterior se abrirá una nueva ventana para configurar el proyecto, en esta ventana se nos pedirá tres cosas, el modelo del Microcontrolador que en este caso es CH32V003 (Recuadro Rojo), el encapsulado que se encuentra fácilmente en la serigrafía del chip que en mi caso es CH32V003F4P6 (Recuadro Verde), y por último el nombre y

ubicación del proyecto (Recuadro Amarillo), cabe mencionar que el nombre no debe contener espacios, una vez configurado lo anterior se dará clic en finish.

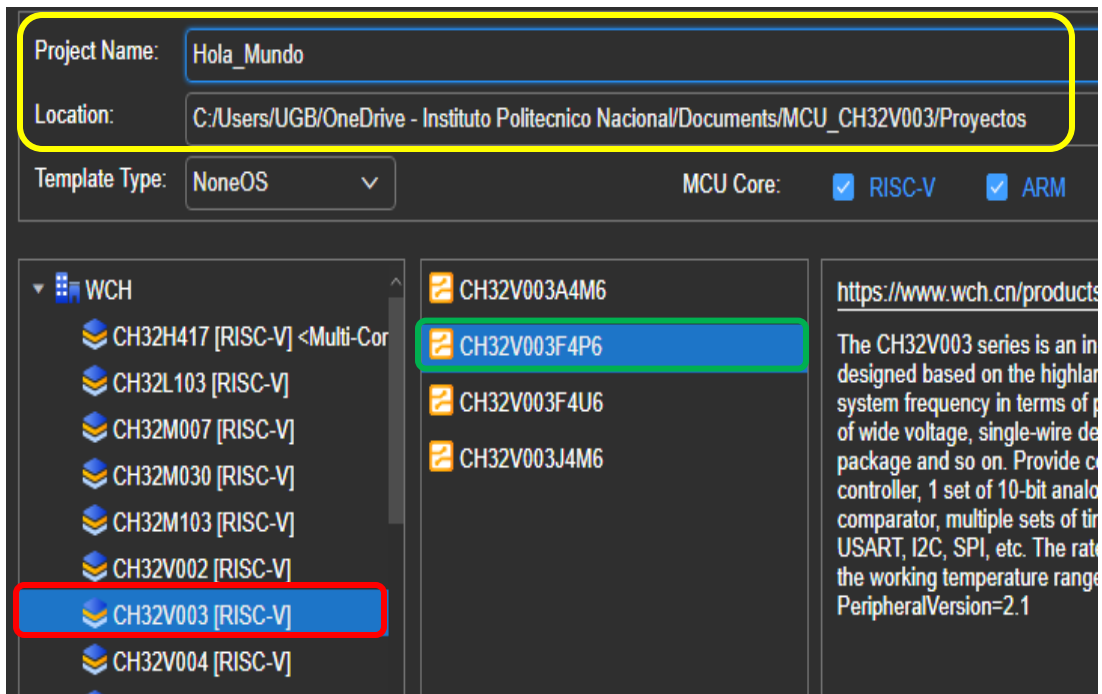


Imagen 2. Nombre, ubicación y selección del chip.

Al finalizar lo anterior la interfaz se observará de la siguiente manera:

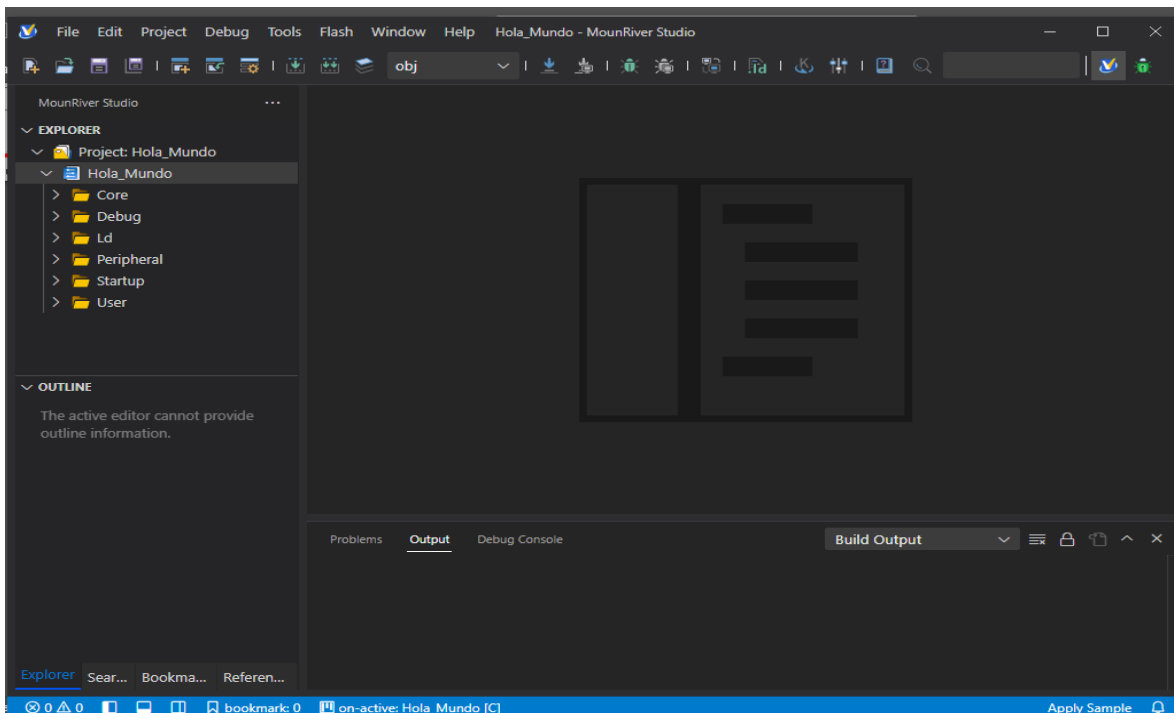


Imagen 3. Interfaz del proyecto.

1.2 Ajuste de proyecto

Antes de ajustar el proyecto procederé a explicar brevemente el contenido de la interfaz.

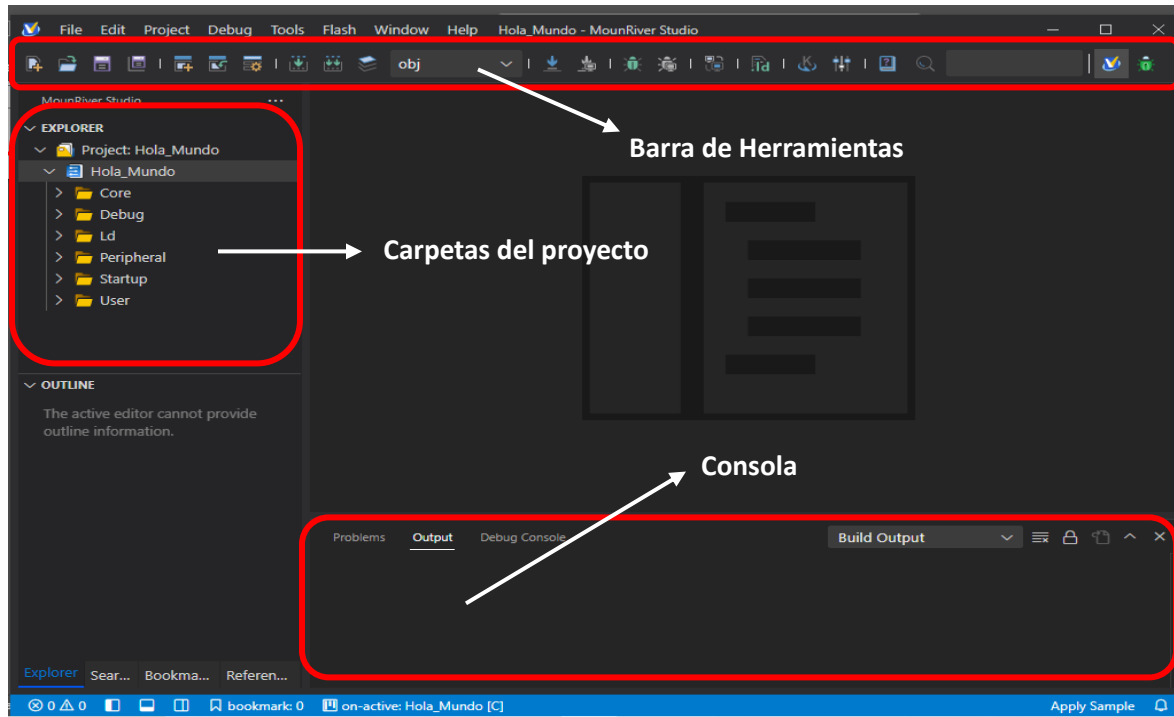


Imagen 4. Partes de la Interfaz.

En la imagen 4 se puede apreciar tres recuadros con un breve nombre, donde:

- **Barra de herramientas:** Contiene los iconos que servirán para compilar, depurar, cargar el archivo compilado al microcontrolador, entre otras opciones necesarias o útiles para el proyecto.
- **Carpetas del proyecto:** Contiene todas las carpetas que el compilador usará para la creación del archivo que será cargado al microcontrolador, en cada carpeta se puede encontrar configuraciones, archivos con extensión C, C++, ASM, h, etc. En nuestro caso la carpeta “User” contiene los archivos para ajustar las configuraciones esenciales del microcontrolador CH32V003F4P6.
- **Consola:** Permite la visualización del estatus del proyecto por ejemplo al compilar, depurar o cargar el firmware al microcontrolador se visualiza errores, tamaño del firmware, variables, etc.

Con lo anterior descrito se procede a configurar el archivo “**system_ch32v00x.c**” y la plantilla “**main.c**”, daremos doble clic izquierdo o clic en la pestaña de expansión de la carpeta User, selecciona dando doble clic izquierdo en los archivos .c anteriormente mencionados, se abrirá el editor mostrando todo el contenido de cada archivo (Recuadro Rojo y verde), esto puede observarse en la imagen 5.

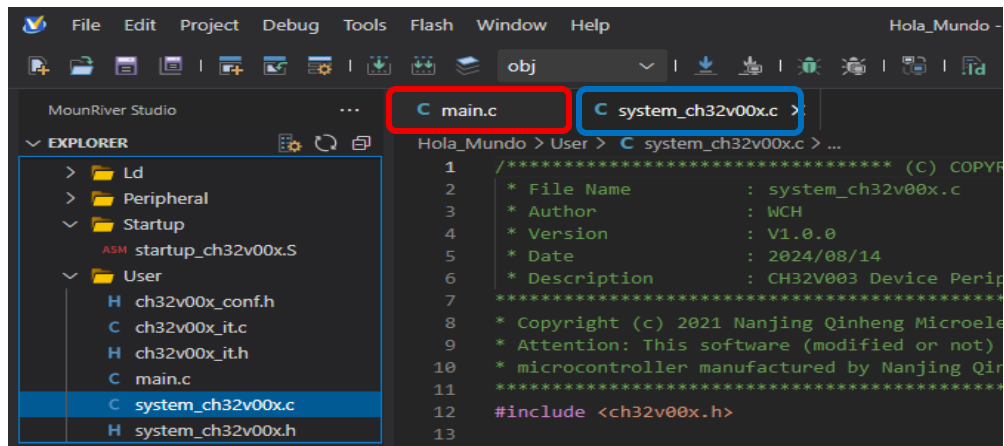


Imagen 5. Editor de código.

Dentro del archivo “**system_ch32v00x.c**” se deberá comentar con un “//” en “**SYSCCLK_FREQ_48MHz_HSE**” y borrar “//” en “**SYSCCLK_FREQ_24MHZ_HSI**” esto puede observarse a detalle en la imagen 6.

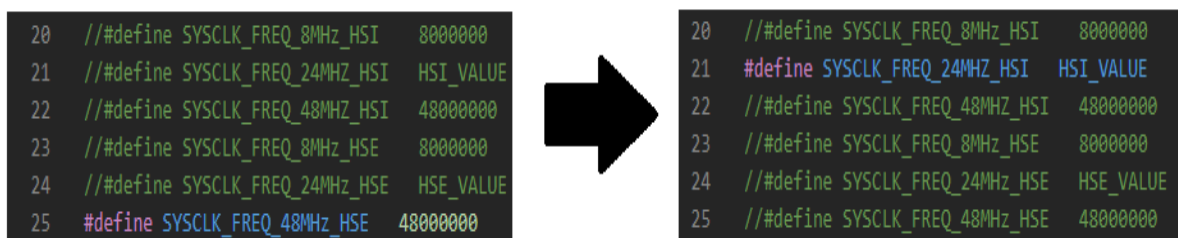


Imagen 6. Selección del oscilador.

Lo anterior ajusta los registros RCC para que el microcontrolador trabaje a una frecuencia de 24 MHz mediante su oscilador interno de tipo RC, cabe mencionar que este ajuste es solo para usuarios que no tengan placas de desarrollo EVT, en dado caso que tengas una placa EVT este paso es opcional ya que cuenta con un oscilador externo.

Para el archivo “**main.c**” solo es borrar su contenido a excepción de lo siguiente:

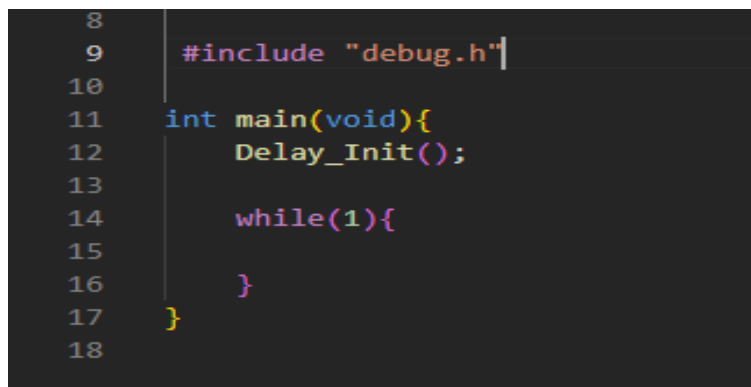


Imagen 7. Modificación del archivo main.c.

Por último, se procede a compilar todo el proyecto seleccionando el icono build Project (Recuadro Rojo) y esperar a que el IDE termine el proceso como se muestra en la imagen 8.

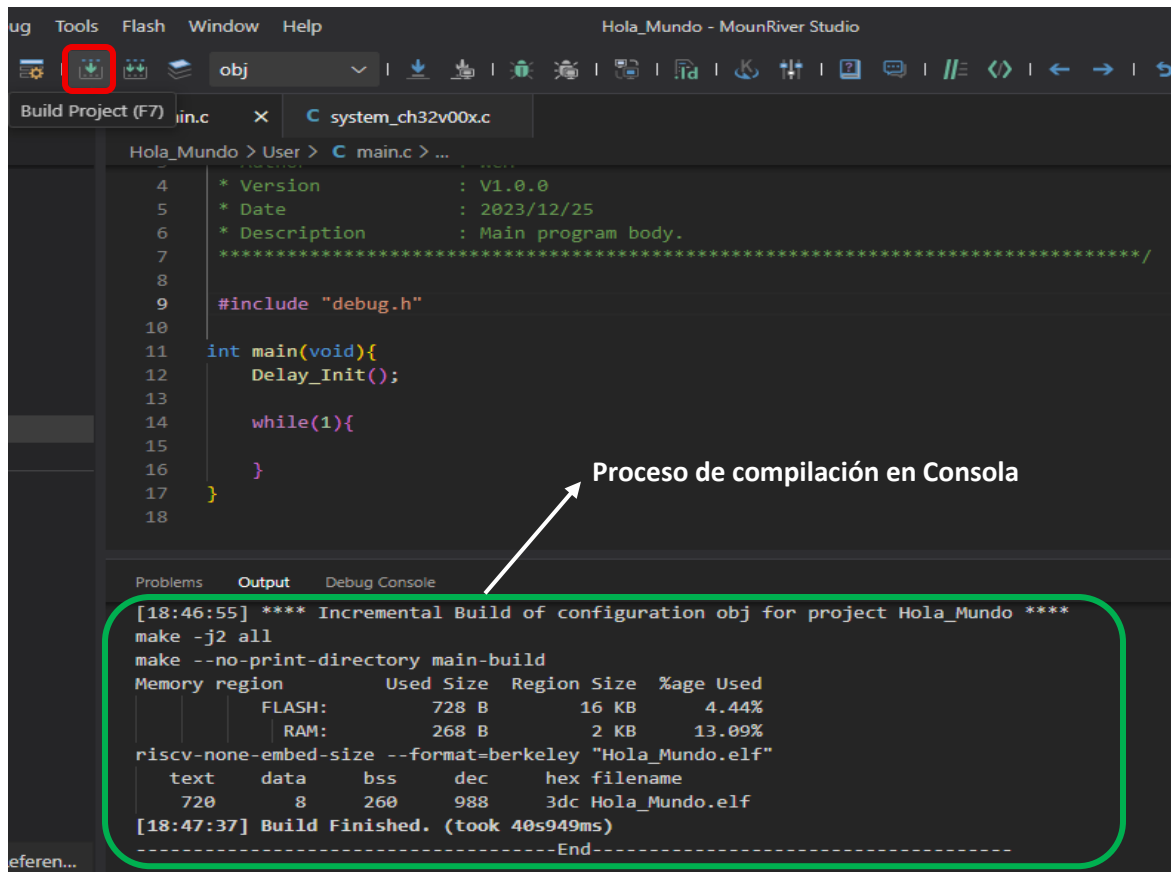


Imagen 8. Nombre y ubicación de main.c.

1.3 Configuración del programador WCH Link-E

Para cargar el firmware del proyecto después de la compilación se requiere de un programador, la empresa WCH cuenta con varios programadores debido a sus diversas familias y por ende hay que tener cuidado ya que no tienen compatibilidad entre ellos, el programador que se usara a partir de este momento es el llamado “**WCH Link-E**”, cabe mencionar que en el mercado hay variantes denominadas como “**WCH LINK-E mini**” que también funcionan, con lo anterior descrito ahora procederé a configurar el programador.

El programador por defecto viene configurado para cargar el firmware en arquitecturas ARM pero en nuestro caso el microcontrolador es el CH32V003 el cual cuenta con una arquitectura Risc-V y por ende dentro del IDE MounRiver Studio 2 se deberá configurar lo siguiente:

1. Conecta el programador a tu computadora y espera a que se carguen los drivers, el programador cuenta con dos leds uno rojo y uno azul, como esta en modo ARM ambos leds permanecerán encendidos, observe la imagen 9.



Imagen 9. Programador en Modo ARM.

2. Como ya se tenía abierto el proyecto que se configuro en el apartado 1.2 entonces se procede a seleccionar la opción **“Flash”** y posteriormente la opción **“Download Configuration”**, esto puede apreciarse claramente en la imagen 10.

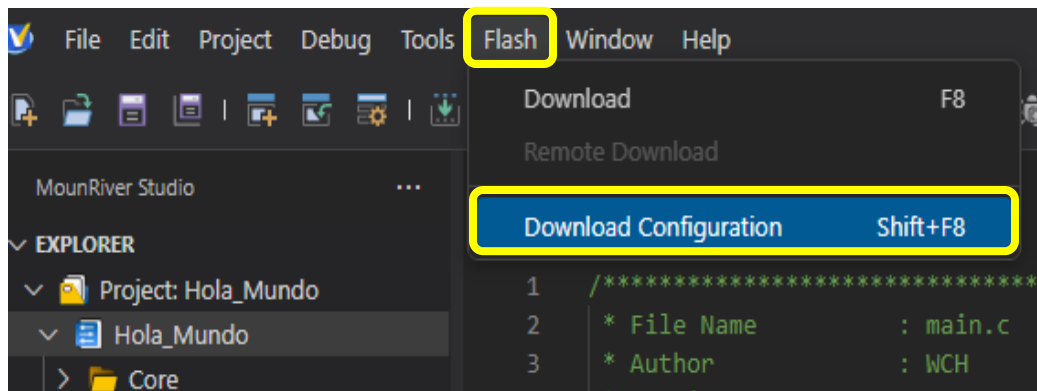


Imagen 10. Opción para abrir la configuración del programador.

3. Una vez realizado el paso anterior se abrirá una ventana la cual permite cambiar de modo ARM a Risc-V y viceversa, con el programador obviamente conectado selecciona **“Query”** (Recuadro Rojo) para detectar el mismo, deberá aparecer ARM (Recuadro Verde) como se muestra en la imagen 11.

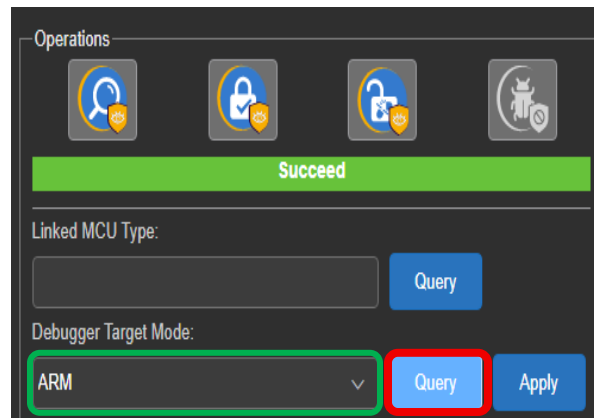


Imagen 11. Detección del Programador.

- Ahora se procede a seleccionar el modo Risc-V ampliando el recuadro que dice ARM del paso 3 y posteriormente dar clic en la opción “Apply” para cargar los cambios al programador.

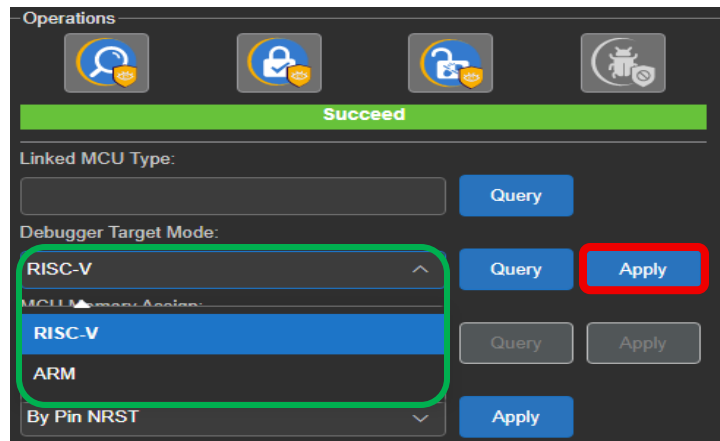


Imagen 12. Selección a Modo Risc-V.

- Una vez realizado lo anterior en el programador “WCH Link-E” debera tener encendido el led de color rojo, eso quiere decir que el ajuste fue exitoso y por ende ya se podrá usar para cargar el firmware al microcontrolador CH32V003.

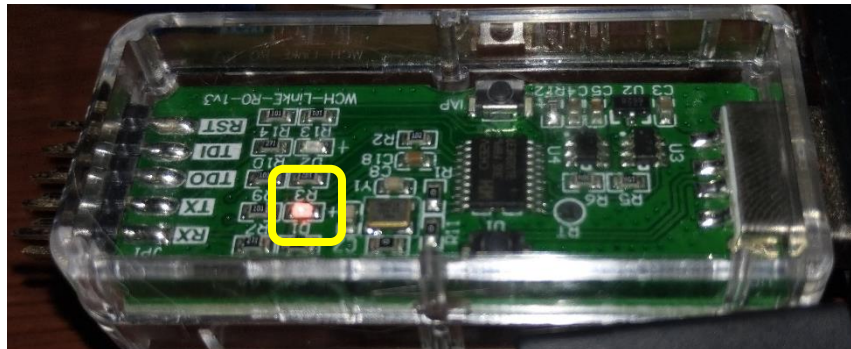


Imagen 13. Programador en Modo Risc-V.

- Una vez que se realizaron todos los pasos anteriores se procede a conectar el Programador y el Microcontrolador como se aprecia en la tabla 1.

Tabla 1. Conexiones entre dispositivos.

Microcontrolador	Programador
VDD (PIN 9)	3V3 (VCC)
VSS (PIN 7)	GND
SWIO (PIN 18)	SWDIO/TMS

2. Salidas

El microcontrolador CH32V003F4P6 tiene a disposición 18 pines que pueden configurarse como salidas, **debe considerar que el pin PD1 no debe usarse para tales fines**, estos 18 pines están distribuidos en tres bloques:

- **Bloque A (Puerto A):** Se compone de los pines PA1 y PA2.
- **Bloque B (Puerto C):** Se compone de los pines PC0 hasta PC7.
- **Bloque D (Puerto D):** Se compone de los pines PD0 hasta PD7.

Debido a la estructura interna de los microcontroladores Risc-V y ARM se deberá usar una serie de pasos, esos pasos consisten en:

- Habilitación del periférico tales como GPIO, DMA, USART, I2C, TIM1, etc.
- Configuración de los pines, esto dependerá del periférico.
- Configuración del periférico.

Estos pasos nos permitirán tener una mejor estructura en el código los cuales se estarán implementando en todos los ejemplos del apartado 4. Los registros asociados que permiten configurar uno o varios pines como salida son:

RCC → APB2PCENR; este registro permite habilitar el reloj que usara el periférico para funcionar correctamente, en este caso puede ser el puerto A, C o D colocando un 1 en el bit “**IOPxEN**”.

3.4.7 PB2 Peripheral Clock Enable Register (RCC_APB2PCENR)

Offset address: 0x18

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	USART1 EN	Reserved	SPI1 EN	TIM1 EN	Reserved	ADC1 EN	Reserved			IOPD EN	IOPC EN	Reserved	IOPA EN	Reserved	AFIO EN

Imagen 14. Registro RCC → APB2PCENR [1].

GPIOx → CFGLR; Este registro permite configurar el modo de salida para un pin en específico ya sea del puerto A, C o D mediante los bits “**MODEy**” cuyas velocidades posibles son b01 (10 MHz), b10 (2 MHz) y b11 (30 MHz), los bits “**CNFy**”, que en este caso son salidas, permiten configurar al pin en modo Push-Pull (b00), Open-Drain (b01) y función multiplexada (b10 y b11), esta última permite enlazar los pines hacia otros periféricos como UART, I2C, etc.

7.3.1.1 Port Configuration Register Low (GPIOx_CFGLR) (x=A/C/D)

Offset address: 0x00

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	

Imagen 15. Registro GPIOx → CFGLR [1].

los registros que permiten el cambio de nivel lógico ya sea 1 o 0 son:

GPIOx → OUTDR; Este registro permite enviar un nivel lógico 1 en los bits “**ODR0**” A “**ODR7**” que son correspondientes a los pines del microcontrolador, por ejemplo; si se requiere que el pin PD5 pase de un nivel 0 lógico a 1 lógico, entonces se debe colocar el bit “**ODR5**” a 1 para que esto se cumpla.

7.3.1.3 Port Output Register (GPIOx_OUTDR) (x=A/C/D)

Offset address: 0x0C

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0

Imagen 16. Registro GPIOx → OUTDR [1].

GPIOx → BCR; Este registro permite enviar un nivel lógico 0 en los bits “**BR0**” A “**BR7**” que son correspondientes a los pines del microcontrolador, por ejemplo; si se requiere que el pin PD2 pase de un nivel 1 lógico a 0 lógico, entonces se debe colocar el bit “**BR2**” a 1 para que esto se cumpla.

7.3.1.5 Port Reset Register (GPIOx_BCR) (x=A/C/D)

Offset address: 0x14

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0

Imagen 17. Registro GPIOx → BCR [1].

GPIOx → BSHR; Este registro permite enviar tanto un nivel lógico 0 en los bits “**BR0**” A “**BR7**” así como también enviar un nivel lógico 1 en los bits “**BS0**” a “**BS7**” que son correspondientes a los pines del microcontrolador, el uso de este registro es mejor ya que con una línea de código se puede ajustar más de un bit con un nivel lógico 1 o 0.

7.3.1.4 Port Reset/Set Register (GPIOx_BSHR) (x=A/C/D)

Offset address: 0x10

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

Imagen 18. Registro GPIOx → BSHR [1].

Todos los registros de este apartado deben usar operadores lógicos (OR, AND, NOT, etc) a excepción del registro “**BSHR**” ya que internamente realiza una operación XOR por hardware.

3. Entradas

Al igual que en el apartado 2 el microcontrolador CH32V003F4P6 tiene a disposición 18 pines que pueden configurarse como Entradas, **debe considerar que el pin PD1 no debe usarse para tales fines**, los registros asociados que permiten configurar uno o varios pines como entrada son:

RCC → APB2PCENR; Este registro permite habilitar el reloj que usara el periférico para funcionar correctamente, en este caso puede ser el puerto A, C o D colocando un 1 en el bit “**IOPxEN**”.

3.4.7 PB2 Peripheral Clock Enable Register (RCC_APB2PCENR)

Offset address: 0x18

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser ved	USAR TI EN	Reser ved	SPI1 EN	TIM1 EN	Reser ved	ADC 1 EN	Reserved			IOPD EN	IOPC EN	Reser ved	IOPA EN	Reser ved	AFIO EN

Imagen 19. Registro RCC → APB2PCENR [1].

GPIOx → CFGLR; Este registro permite configurar el modo de entrada para un pin en específico ya sea del puerto A, C o D mediante los bits “**MODEy**” en una configuración 0b00 (Input Mode), los bits “**CNFy**”, permiten configurar al pin en modo de entrada analógica (b00), entrada Flotante (b01) y por último como entrada con resistencias Pull-Up y Pull-Down (0b10), este último modo requiere el uso del registro “**OUTDR**” para habilitar la resistencia.

7.3.1.1 Port Configuration Register Low (GPIOx_CFGLR) (x=A/C/D)

Offset address: 0x00

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	

Imagen 20. Registro GPIOx → GFGLR [1].

GPIOx → OUTDR; Este registro permite usar la resistencia interna como Pull-UP asignando un 1 o como Pull-Down asignando un 0 en los bits “**ODR0**” A “**ODR7**”.

7.3.1.3 Port Output Register (GPIOx_OUTDR) (x=A/C/D)

Offset address: 0x0C

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0

Imagen 21. Registro GPIOx → OUTDR [1].

4. Ejemplos

4.1 Hola mundo!

El hola mundo de los microcontroladores es el generar la activación y desactivación de un led conectado en un pin que en este caso será el pin PC5, a partir de este momento todos los ejemplos van a contener diagrama electrónico, código y descripción.

Montar el circuito de la imagen 22 con la ayuda de un protoboard.

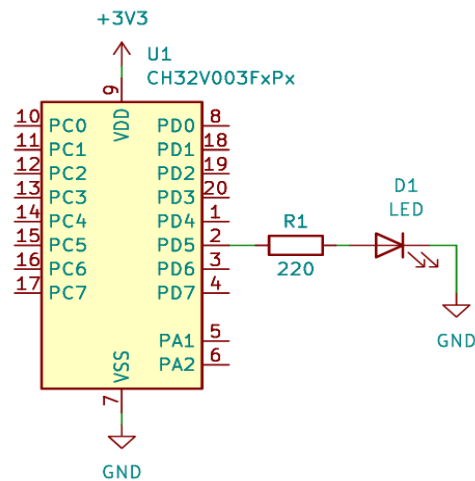


Imagen 22. Diagrama electrónico.

Dentro del archivo main.c copia y pega el siguiente código y posteriormente procede a compilar el proyecto([Link](#)).

```
//Codigo #1: Hola Mundo
#include "debug.h"

#define LED_ON (GPIO_D -> BSHR = GPIO_BSHR_BS5) // Pin PD5 a 1 logico
#define LED_OFF (GPIO_D -> BSHR = GPIO_BSHR_BR5) // Pin PD5 a 0 logico

int main(void){
    Delay_Init(); // Inicializa los tiempos us y ms

    /* ACTIVACION DE PERIFERICOS */
    RCC -> APB2PCENR |= RCC_IOPDEN; // Se habilita el reloj del puerto D

    /* CONFIGURACION DE LOS PINES DEL PUERTO D */
    GPIO_D -> CFGLR &= ~(GPIO_CFGLR_MODE5 | GPIO_CFGLR_CNF5); // Borra configuraciones iniciales
    GPIO_D -> CFGLR |= GPIO_CFGLR_MODE5; // PD5 como salida a 30 MHz en modo Push-Pull

    while(1){
        LED_ON; // PD5 en 1 logico
        Delay_Ms(500);
    }
}
```

```

    LED_OFF; // PD5 en 0 logico
    Delay_Ms(500);
}
}

```

A partir de aquí cada código de este apartado se dividirá en tres secciones para describir su funcionamiento (Cabecera, Inicialización de periféricos y ciclo infinito).

Cabecera:

Está constituida por el archivo “**debug.h**” cuyo contenido es generar los tiempos us y ms usando el periférico “**SysTick**”, también contiene el archivo “**ch32v00x.h**” el cual contiene todos los registros del microcontrolador CH32V003, por otra parte, se crearon dos etiquetas, la primera es “**LED_ON**” la cual permite asignar un 1 lógico en el pin PD5 y la segunda etiqueta es “**LED_OFF**” la cual asigna un 0 lógico en el pin PD5, esto es gracias al registro “**BSHR**”, obsérvese que se está asignando directamente el bit correspondiente al pin y que no usa operadores OR(|) y/o AND(&) ya que el registro “**BSHR**” genera un XOR por hardware.

Inicialización de periféricos:

Esta parte se ubica después de la función main y antes del ciclo infinito (“**while(1)**”), observe imagen 23.

```

14  int main(void){
15      Delay_Init(); // Inicializa los tiempos us y ms
16
17      /* ACTIVACION DE PERIFERICOS */
18      RCC -> APB2PCENR |= RCC_IOPDEN; // Se habilita el reloj del puerto D
19
20      /* CONFIGURACION DE LOS PINES DEL PUERTO D */
21      GPIOD -> CFGLR &= ~(GPIO_CFGLR_MODE5 | GPIO_CFGLR_CNFS); // Borra configuraciones iniciales
22      GPIOD -> CFGLR |= GPIO_CFGLR_MODE5; // PD5 como salida a 30 MHz en modo Push-Pull
23
24      while(1){

```

Imagen 23. Sección de inicialización.

El contenido en esta sección es inicializar los registros asociados a los periféricos del microcontrolador como puertos, interrupciones, timers, protocolos, etc, en este caso se realizan tres configuraciones, la primera es “**Delay_init()**” la cual configura el periférico “**SysTick**” para generar retrasos en el orden de los microsegundos(us) y milisegundos(ms), la segunda configuración es el registro “**APB2PCENR**” el cual permite activar el reloj del periférico “**GPIOD**” para su correcto funcionamiento y por último la configuración del periférico “**GPIOD**” el cual usa el registro “**CFGLR**” para que el pin PD5 funcione como una salida en modo Push-Pull a una velocidad máxima de 30 MHz.

En cada registro del microcontrolador existe una configuración por defecto la cual puede ser observada en su datasheet, imagen 24, por lo cual siempre que se requiera usar o configurar un registro debe borrar sus configuraciones iniciales.

Bit	Name	Access	Description	Reset value
[31:30] [27:26] [23:22] [19:18] [15:14] [11:10] [7:6] [3:2]	CNFy[1:0]	RW	(y=0-7), the configuration bits for port x, by which the corresponding port is configured. When in input mode (MODE=00b). 00: Analog input mode. 01: Floating input mode. 10: With pull-up and pull-down mode. 11: Reserved. In output mode (MODE>00b). 00: Universal push-pull output mode. 01: Universal open-drain output mode. 10: Multiplexed function push-pull output mode. 11: Multiplexing function open-drain output mode.	01b
[29:28] [25:24] [21:20] [17:16] [13:12] [9:8] [5:4] [1:0]	MODEy[1:0]	RW	(y=0-7), port x mode selection, configure the corresponding port by these bits. 00: Input mode. 01: Output mode, maximum speed 10MHz; 10: Output mode, maximum speed 2MHz. 11: Output mode, maximum speed 30MHz.	00b

Imagen 24. Valor por defecto en registro CFGLR [1].

Ciclo infinito:

Esta sección tiene como objetivo repetir una parte del código infinitamente, en este caso el código que está dentro de los corchetes mantiene al pin PD5 en 1 lógico durante 500 ms y posteriormente en 0 lógico durante otros 500 ms así generando una oscilación de 1 segundo.

4.2 Encender led mediante una entrada

Para este ejemplo implica usar una entrada digital en el pin PC0 cuyo objetivo es encender un led en la salida PD2 cada vez que está presente un nivel de 1 lógico.

Montar el circuito de la imagen 25 con la ayuda de un protoboard.

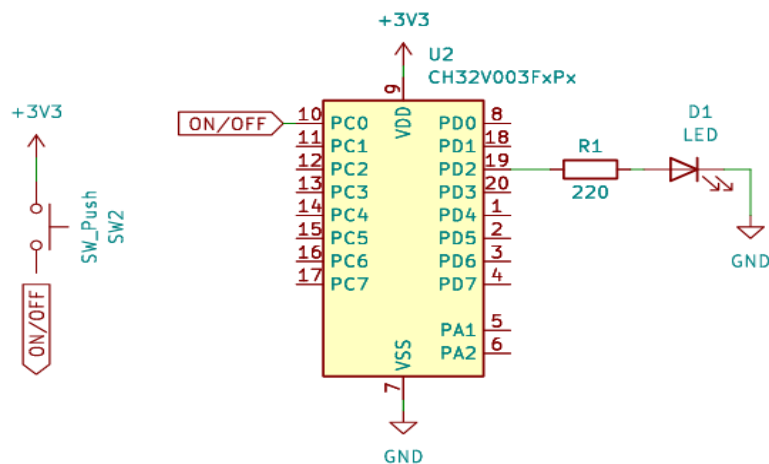


Imagen 25. Diagrama electrónico.

Dentro del archivo main.c copia y pega el siguiente código y posteriormente procede a compilar el proyecto([Link](#)).

```
//Codigo #2: Activacion de LED mediante una entrada
#include "debug.h"

#define LED_ON (GPIOC -> BSHR = GPIO_BSHR_BS2) // Pin PD2 a 1 logico
#define LED_OFF (GPIOC -> BSHR = GPIO_BSHR_BR2) // Pin PD2 a 0 logico

int main(void){

    /* ACTIVACION DE PERIFERICOS */
    RCC -> APB2PCENR |= RCC_IOPCEN | RCC_IOPDEN; // Habilitacion del reloj en puerto C y D

    /* CONFIGURACION DE LOS PINES DEL PUERTO C */
    GPIOC -> CFGLR &= ~(GPIO_CFGLR_MODE0 | GPIO_CFGLR_CNF0); // Borra configuraciones iniciales
    GPIOC -> CFGLR |= GPIO_CFGLR_CNF0_1; // PC0 como entrada en modo Pull-Down por defecto

    /* CONFIGURACION DE LOS PINES DEL PUERTO D */
    GPIOD -> CFGLR &= ~(GPIO_CFGLR_MODE2 | GPIO_CFGLR_CNF2); // Borra configuraciones iniciales
    GPIOD -> CFGLR |= GPIO_CFGLR_MODE2; // PD2 como salida a 30 MHz en modo Push-Pull

    while(1){
        /* ACTIVA EL LED SI HAY UN 1 LOGICO EN PC0 */
        if ( GPIOC -> INDR & GPIO_INDR_IDR0){
            LED_ON;
        }
        else {
            LED_OFF;
        }
    }
}
```

Cabecera:

De igual manera que el código #1, esta parte se compone por el archivo “**debug.h**” cuyo contenido es generar los tiempos us y ms usando el periférico “**SysTick**”, también se crearon las mismas etiquetas como “**LED_ON**” la cual permite asignar un 1 lógico en el pin PD5 y “**LED_OFF**” la cual asigna un 0 lógico en el pin PD5.

Inicialización de periféricos:

Primeramente, se habilita los periféricos “**GPIOC**” y “**GPIOD**” usando el registro “**APB2PCENR**”, después se configura al pin PC0 como una entrada con resistencia de Pull-Down, cabe mencionar que el registro “**OUTDR**” por defecto tiene un valor de 0 por defecto por lo tanto la resistencia es Pull-Down, por último, se configura al pin PD2 como salida en modo Push-Pull a una velocidad máxima de 30 MHz.

Ciclo infinito:

Para este ejemplo se implementa el uso de un condicional if el cual permite determinar si en el pin PC0 hay un nivel 1 lógico para activar la salida, de lo contrario la salida estará en un nivel 0 lógico. El funcionamiento es simple, el condicional if verifica a cada momento si el bit “**IDR0**” dentro del registro “**INDR**” tiene un nivel de 1 lógico mediante una operación &(AND), como el pin PC0 está

directamente conectado a GND mediante la resistencia de Pull-Down el condicional if determina que el resultado de operación AND es falso y por ende el pin PD2 estará en un nivel de 0 lógico todo el tiempo, cuando en el pin PC0 se le suministra un voltaje de 3.3 volts mediante el push button el condicional if determina que la operación and es verdadera por lo que en el pin PD2 estará en un nivel de 1 lógico.

4.3 Conteo de pulsos con salida en binario

En este ejemplo se contarán pulsos mediante el pin PD4 cuando esté presente un nivel de 1 lógico, el conteo podrá ser observado mediante un arreglo de 7 leds conectados en el puerto C donde el bit menos significativo es el pin PC0 y el bit más significativo es el pin PC6 dando un conteo de 0 a 127.

Montar el circuito de la imagen 26 con la ayuda de un protoboard.

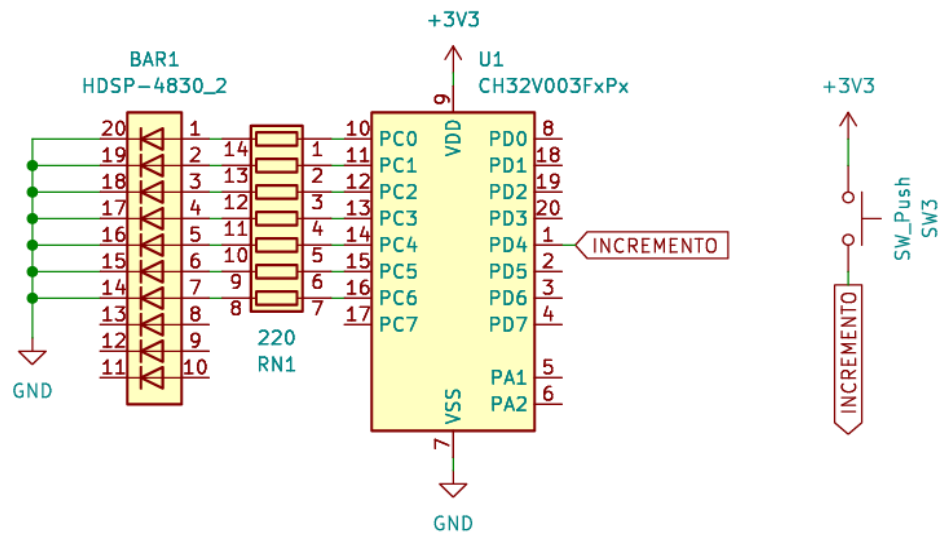


Imagen 26. Diagrama electrónico.

Dentro del archivo main.c copia y pega el siguiente código y posteriormente procede a compilar el proyecto ([Link](#)).

```
//Codigo #3: Conteo de pulsos con salida en binario
#include "debug.h"

int main(void){
    /* ACTIVACION DE PERIFERICOS */
    RCC -> APB2PCENR |= RCC_IOPCEN | RCC_IOPDEN; // Se habilita el reloj del puerto C y D

    /* CONFIGURACION DE LOS PINES DEL PUERTO C */
    GPIOC -> CFGLR &= ~(GPIO_CFGLR_MODE | GPIO_CFGLR_CNF); // Borra configuraciones iniciales
    GPIOC -> CFGLR |= GPIO_CFGLR_MODE; // Puerto C como salida a 30 MHz en modo Push-Pull

    /* CONFIGURACION DE LOS PINES DEL PUERTO D */
    GPIOD -> CFGLR &= ~(GPIO_CFGLR_MODE4 | GPIO_CFGLR_CNF4); // Borra configuraciones iniciales
    GPIOD -> CFGLR |= GPIO_CFGLR_CNF4_1; // PD4 como entrada en modo pull-down por defecto
```



```

uint8_t Cuenta=0; // variable local de control

while(1){
    /* SE DETERMINA QUE EL PIN DE ENTRADA SEA PD4 */
    if ( GPIOD -> INDR & GPIO_INDR_IDR4 ) {
        while (GPIOD -> INDR & GPIO_INDR_IDR4); // Antirrebote
        GPIOC -> OUTDR = Cuenta++; // Se envía el valor de cuenta hacia el puerto C
    }
}

```

Inicialización de periféricos:

Primeramente, se habilita los periféricos **“GPIOC”** y **“GPIOD”** usando el registro **“APB2PCENR”**, después se configura todo el puerto C como salida en modo Push-Pull a una velocidad máxima de 30 MHz, cabe mencionar que cuando se requiere a todo el puerto como salida o como entrada los bits **“MODEx”** y **“CNFy”** no deben contener la referencia de **“x”** y **“y”** la cual indica el número del pin, después se configura el pin PD4 como una entrada con resistencia en modo Pull-Down por defecto y por último se crea una variable local de ocho bits sin signo (**“uint8_t”**) con el nombre de **“cuenta”**, es importante conocer el tipo de la variable ya que esta nos permitirá trabajar con un rango en específico ahorrando memoria flash dentro del microcontrolador.

Ciclo infinito:

En este ejemplo se implementa el uso del condicional if el cual permite determinar si en el pin PD4 hay un nivel de 1 lógico o no. El funcionamiento es simple, el condicional if verifica a cada momento si el bit **“IDR4”** dentro del registro **“INDR”** tiene un nivel de 1 lógico mediante una operación **&(AND)**, como el pin PD4 está directamente conectado a GND mediante la resistencia de Pull-Down en todo momento el condicional if determina que el resultado de la operación AND es falso y por ende la variable **“cuenta”** no incrementara, cuando el pin PD4 se le suministra un voltaje de 3.3 volts mediante el push button, el condicional if determina que la operación AND es verdadera por lo que la variable **“cuenta”** incrementara en una unidad y a su vez se enviara el resultado hacia el puerto C mediante el registro **“OUTDR”**, cabe mencionar que el incrementar una unidad es debido al operador **“++”**, este operador puede ir antes o después de una variable pero hay que tener mucho cuidado en su uso ya que cualquiera de las opciones puede causar algunos problemas en implementaciones complejas, es por ello que mencionare y explicare su uso en el **“Anexo A”**.

El uso de conmutadores mecánicos como el push button ocasionan rebotes al momento de pulsar y también al momento de soltar (observe imagen 27), estos rebotes son del rango de los microsegundos los cuales pueden ser interpretados por el microcontrolador generando múltiples pulsos sobre todo en variables de conteo, por tal motivo se usó un condicional while el cual bloquea infinitamente el recorrido del programa hasta que el pin, en este caso PD4, pase de 1 a 0 lógico, cabe mencionar que para velocidades superiores a los 50 MHz la implementación anteriormente mencionada deberá estar seguida de un delay de al menos 10 us después del condicional while para evitar problemas.

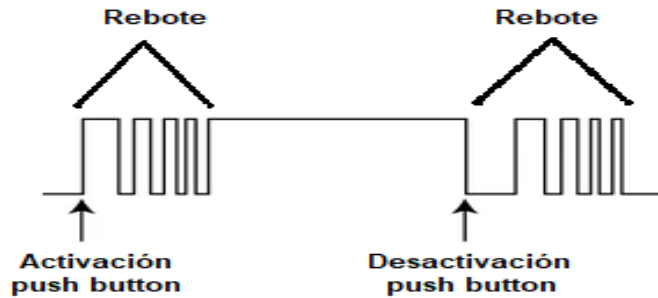


Imagen 27. Rebotes de un push button.

4.4 Conteo de 0 a 99 con display de siete segmentos

En este ejemplo se implementará un contador automático de 0 a 99 donde cada número podrá ser observado mediante dos display de siete segmentos cada 250 ms, el control de los display es mediante la multiplexación la cual permite enviar mediante un solo puerto las unidades y decenas en un intervalo de tiempo, por ejemplo, supongamos que tenemos la variable **“conteo”** con un valor de 23 entonces primeramente se envía el valor 2 codificado durante 1 ms para el display A y posteriormente se envía el valor 3 codificado durante 1 ms para el display B, este tipo de técnica ocasiona que visualmente se observe el valor 23 de manera estática y además alarga la vida de los leds ya que no están encendidos todo el tiempo.

Montar el circuito de la imagen 28 con la ayuda de un protoboard.

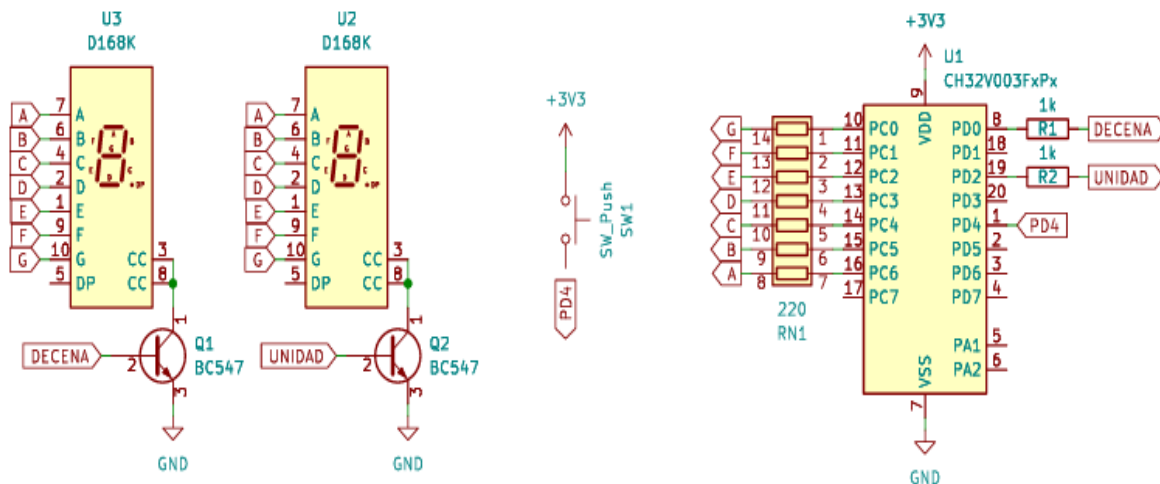


Imagen 28. Diagrama electrónico.

Dentro del archivo main.c copia y pega el siguiente código y posteriormente procede a compilar el proyecto ([Link](#)).

```
//Codigo #4: Conteo de 0 a 99 con display de siete segmentos
#include "debug.h"

const uint8_t Display[]={ 0x7E, 0x30, 0x6D, 0x79, 0x33, 0x5B, 0x5F, 0x70, 0x7F, 0x73 };
//codificación de display catodo comun
```

```

int main(void){
    Delay_Init();

    /* CONFIGURACION DE PERIFERICOS */
    RCC -> APB2PCENR |= RCC_IOPCEN | RCC_IOPDEN; // Se habilita el reloj de puerto C y D

    /* CONFIGURACION DE LOS PINES DEL PUERTO C */
    GPIOC -> CFGLR &= ~(GPIO_CFGLR_CNF | GPIO_CFGLR_MODE); // Reset en todo el puerto C
    GPIOC -> CFGLR |= GPIO_CFGLR_CNF | GPIO_CFGLR_MODE; // Puerto C como salida push-pull a
30MHz

    /* CONFIGURACION DE LOS PINES DEL PUERTO D */
    GPIOD -> CFGLR &= ~(GPIO_CFGLR_CNF0 | GPIO_CFGLR_MODE0 | GPIO_CFGLR_CNF2 | GPIO_CFGLR_MODE2 |
GPIO_CFGLR_CNF4 | GPIO_CFGLR_MODE4 ); // Se borra configuracion por
defecto en PD0, PD2 y PD4
    GPIOD -> CFGLR |= GPIO_CFGLR_CNF4_1 | GPIO_CFGLR_MODE0 | GPIO_CFGLR_MODE2; // PD4 como
entrada en modo pull-down por defecto, PD0 y PD2 como salida push-pull a 30MHz

    uint8_t Cuenta=0, Flag_Cuenta=0; // Variables locales de control

    while(1){
        /* SE DETERMINA QUE EL PIN DE ENTRADA SEA PD4*/
        if ( (Flag_Cuenta==0) && (GPIOD -> INDR & GPIO_INDR_IDR4)){
            while (GPIOD -> INDR & GPIO_INDR_IDR4); // Antirrebote
            Flag_Cuenta = 1;
        }

        /* MULTIPLEXACION DE LOS DATOS */
        if (Flag_Cuenta == 1) {
            for (uint8_t i=0; i< 31; i++ ) { //delay 248 ms por cuenta
                GPIOC -> OUTDR = Display[Cuenta/10]; // DECENAS
                GPIOD -> BSHR = GPIO_BSHR_BS0 | GPIO_BSHR_BR2;
                Delay_Ms(4);

                GPIOC -> OUTDR = Display[Cuenta%10]; // UNIDADES
                GPIOD -> BSHR = GPIO_BSHR_BS2 | GPIO_BSHR_BR0;
                Delay_Ms(4);
            }
            Cuenta++;
        }

        /* REINICIO DE VARIABLES*/
        if (Cuenta>99) {
            Cuenta = Flag_Cuenta = 0;
            GPIOD -> BSHR = GPIO_BSHR_BR0 | GPIO_BSHR_BR2;
        }
    }
}

```

Cabecera:

En este ejemplo se está implementando una variable global constante con nombre “Display” el cual es un arreglo de diez posiciones cuyo contenido es la codificación que representaran los segmentos del display del 0 al 9, esto puede ser observado en la tabla 2, cabe mencionar que hasta el momento se ha trabajado con variables locales y la pregunta es, ¿cuál es la diferencia de una variable local y una global constante?, la diferencia es que una variable “**local**” solo existe y podrá ser usada dentro de una función específica mientras que una variable “**global**” constante puede ser usada en cualquier parte del código con la restricción que no podrá ser modificada.

Tabla 2. Codificación display siete segmentos.

Número	A	B	C	D	E	F	G	Hexadecimal
	(PC6)	(PC5)	(PC4)	(PC3)	(PC2)	(PC1)	(PC0)	
0	1	1	1	1	1	1	0	7E
1	0	1	1	0	0	0	0	30
2	1	1	0	1	1	0	1	6D
3	1	1	1	1	0	0	1	79
4	0	1	1	0	0	1	1	33
5	1	0	1	1	0	1	1	5B
6	1	0	1	1	1	1	1	5F
7	1	1	1	0	0	0	0	70
8	1	1	1	1	1	1	1	7F
9	1	1	1	0	0	1	1	73

En la tabla dos se puede observar una serie de columnas donde el “**Número**” es el valor que podrá observarse en el display, las letras de la “**A**” hasta la “**G**” representan los segmentos del display que en este caso es de cátodo común eso quiere decir que los leds internamente tienen conexión común en su pin negativo (cátodo) mientras que su pin positivo (ánodo) está disponible para ser usado por el microcontrolador (observe imagen 29), cabe mencionar que aparte de representar los segmentos del display también representan los pines del PC6 hasta PC0, por último se tiene la columna “hexadecimal” la cual representa el binario que se forma en cada fila.

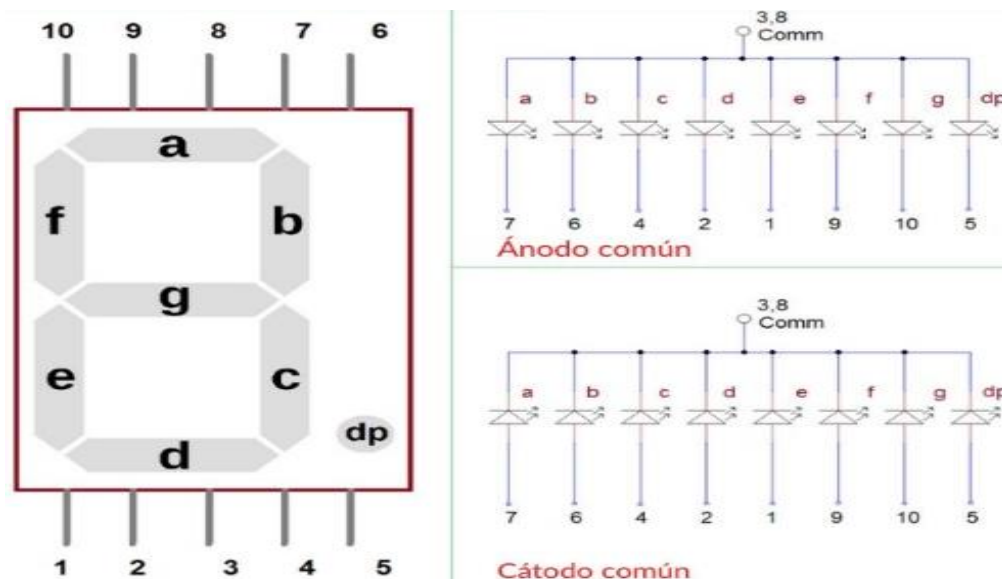


Imagen 29. Diagrama display de siete segmentos.

Inicialización de periféricos:

Primeramente, se habilita los periféricos “**GPIOC**” y “**GPIOD**” usando el registro “**APB2PCENR**”, después se configura todo el puerto C como salida en modo Push-Pull a una velocidad máxima de 30 MHz, este puerto permitirá enviar el valor de las unidades y decenas para el display de siete segmentos, posteriormente se configura el puerto D donde el pin PD0 y PD2 serán salidas en modo Push-Pull a una velocidad máxima de 30 MHz, estos pines serán usados para multiplexar los display mientras que el pin PD4 se está configurando como una entrada con resistencia en modo Pull-Down por defecto, por último se crean dos variables locales de ocho bits sin signo (“**uint8_t**”) donde la variable “**cuenta**” tiene como objetivo almacenar el valor de 0 a 99 mientras que la variable “**Flag_cuenta**” impide que cualquier pulso del pin PD4 interrumpa con el conteo.

Ciclo infinito:

Para esta sección de código se implementa toda la lógica de control que en este caso es la multiplexación, la lectura del nivel lógico en el pin PD4, etc, etc. Para una mejor descripción procederé a enumerar el código como se muestra en el siguiente recuadro:

```
while(1){
    /* SE DETERMINA QUE EL PIN DE ENTRADA SEA PD4*/
    if ( (Flag_Cuenta==0) && (GPIOD -> INDR & GPIO_INDR_IDR4)){
        while (GPIOD -> INDR & GPIO_INDR_IDR4); // Antirrebote
        Flag_Cuenta = 1;
    }

    /* MULTIPLEXACION DE LOS DATOS */
    if (Flag_Cuenta == 1) {
        for (uint8_t i=0; i< 31; i++) { //delay 248 ms por cuenta
            GPIOC -> OUTDR = Display[Cuenta/10]; // DECENAS
            GPIOD -> BSHR = GPIO_BSHR_BS0 | GPIO_BSHR_BR2;
            Delay_Ms(4);

            GPIOC -> OUTDR = Display[Cuenta%10]; // UNIDADES
            GPIOD -> BSHR = GPIO_BSHR_BS2 | GPIO_BSHR_BR0;
            Delay_Ms(4);
        }
        Cuenta++;
    }

    /* REINICIO DE VARIABLES*/
    if (Cuenta>99) {
        Cuenta = Flag_Cuenta = 0;
        GPIOD -> BSHR = GPIO_BSHR_BR0 | GPIO_BSHR_BR2;
    }
}
}
```

En la sección marcada como (1) se tiene el primer condicional if el cual compara en todo momento que el nivel lógico del pin PD4 sea 1 y a su vez que la variable “**Flag_cuenta**” tenga un valor de cero, si ambas condiciones son verdaderas entonces entra y se le asigna un valor de uno a la variable “**Flag_Cuenta**”, esta asignación permite que el segundo condicional if de la sección (2) se mantenga en

estado verdadero para generar la multiplexación y el incremento de la variable **“cuenta”**.

En la sección marcada como (2) se tiene un condicional if cuyo objetivo es generar la multiplexación y el conteo de la variable **“cuenta”** siempre que la variable **“Flag_cuenta”** tenga un valor de uno, dentro de este if se tiene un condicional for cuyo objetivo es repetir 31 veces el envío de las decenas y unidades hacia el puerto C cada 8 milisegundos, el funcionamiento es el siguiente:

- Se envía el valor de la decena hacia el puerto C usando el registro **“OUTDR”**, en el puerto D se envía un 1 lógico en el pin PD2 y un 0 lógico en el pin PD0 mediante el registro **“BSHR”**, lo anterior se mantiene durante 4 ms.
- Se envía el valor de la unidad hacia el puerto C usando el registro **“OUTDR”**, en el puerto D se envía un 0 lógico en el pin PD2 y un 1 lógico en el pin PD0 mediante el registro **“BSHR”**, lo anterior se mantiene durante 4 ms.
- Después que se envió 31 veces la decena y unidad se incrementa la variable **“cuenta”** en una unidad repitiendo el ciclo hasta llegar a 99.

Lo anterior genera una multiplexación de cada display a una frecuencia de 250 Hz con un tiempo entre cada conteo de aproximadamente 250 ms, para determinar las veces que se debe repetir la multiplexación de los display es mediante la siguiente ecuación:

$$N_{veces} = \frac{\text{Tiempo a llegar}}{\text{Tiempo multiplexado}}$$

Dando valores se tiene que el ciclo for debe repetirse:

$$N_{veces} = \frac{250 \text{ ms}}{8 \text{ ms}} = 31.25$$

Con lo anterior se justifica por qué la variable i dentro del ciclo for debe alcanzar el valor de 31, por otra parte, el código que se encuentra dentro de los corchetes del ciclo for es básicamente asignar el valor codificado de las decenas y unidades en el puerto C donde cada display se activara gracias a las salidas de los pines PD0 y PD2 del puerto D durante 8 ms.

La última sección marcada como (3) cumple la función de parar el conteo asignando un cero en la variable **“Flag_cuenta”** y a su vez asigna a la variable **“cuenta”** a su valor por defecto (cero) así como también en los pines PD0 y PD2 se mantienen en un nivel 0 lógico para que en el display no se muestre nada

5 FUENTES DE INFORMACIÓN

[1] Datasheet WCH32V003, WCH. [CH32V003RM.PDF - Nanjing Qinheng Microelectronics Co., Ltd.](#)

[2] Ejemplos WCH32V003, UGB_Dev. [UGB-Dev/Ejemplos_CH32V003](#)

Anexo A

Tabla 3.- Operadores Lógicos.

Operador	Operación	Descripción
OR ()	0111 0010 = 0111	El resultado es 0 si ambos bits son 0, de lo contrario el resultado es 1
AND (&)	0111 & 0010 = 0010	El resultado es 1 si ambos bits son 1, de lo contrario el resultado es 0
XOR (^)	0111 ^ 0010 = 0101	El resultado es 0 si ambos bits son 1 o 0, de lo contrario el resultado es 1
NOT (~)	~ 0101 = 1010	El resultado es 0 si el bit es 1 y viceversa
otras		
Desplazamiento a la izquierda (<<)	01010101 << 3 = 10101000	Se desplaza a la izquierda el número de bits especificado a la derecha del operador
Desplazamiento a la derecha (>>)	01010101 >> 3 = 00001010	Se desplaza a la derecha el número de bits especificado a la derecha del operador

La operación “++” es una manera fácil de representar un incremento en una unidad y dependerá si esta antes o después de una variable o registro si hablamos de microcontroladores, por ejemplo, suponiendo que requerimos que en el puerto D se incremente una unidad cada 500 ms mediante la variable “**CONT**” con un valor inicial en 2.

GPIOD → OUTDR = ++ CONT; (1)

Lo que se realiza en el código anterior es incrementar en una unidad la variable “**CONT**” y asigna ese valor al registro “**OUTDR**”.

GPIOD → OUTDR = CONT ++; (2)

Lo que se realiza en el código anterior es asignar el valor de la variable “**CONT**” en el registro “**OUTDR**” y posteriormente la variable “**CONT**” incrementara en una unidad.

La diferencia es clara, la salida en el puerto D para el código (1) empezara desde 3 mientras que en el código (2) empezara desde 2, esto se debe tener mucho en cuenta al usarse en arreglos o en UART.